SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

CO_3-AT_1-EXPERIMENTS

Course Code: CSA0609

Course Title: Design and Analysis of Algorithms

Name: Jeeva

Reg No: 192421334

Question 32: Evaluate the performance of Quick Sort using median-of-three pivot selection on large datasets. Compare execution time with first and last element pivot strategies. Record and plot the results for different input distributions. Interpret how median selection reduces the likelihood of worst-case scenarios and justify its effectiveness.

AIM

To evaluate the performance of Quick Sort using **median-of-three pivot selection** and compare its execution time with Quick Sort using **first-element** and **last-element** pivot strategies for different input distributions.

ALGORITHM

1. Generate datasets of different sizes.
2. Create different input distributions:
    - Random
    - Sorted
    - Reverse

    sorted
3. Implement Quick Sort using the **first element** as pivot.
4. Implement Quick Sort using the **last element** as pivot.
5. Implement Quick Sort using the **median of the first, middle, and last elements** as pivot.
6. Measure the execution time of each method.

7. Repeat the experiment for different input sizes.
8. Store the execution times.
9. Plot the execution times using a graph.
10. Compare the results and determine which pivot strategy performs better.

PSEUDOCODE

```
QUICKSORT(array, low, high, pivot_type)
   if low < high:
      pivot = SELECT_PIVOT(array, low, high, pivot_type)
Swap pivot with last element        p = PARTITION(array,
low, high)
      QUICKSORT(array, low, p-1, pivot_type)
      QUICKSORT(array, p+1, high, pivot_type)
SELECT_PIVOT(array, low, high, type)    if
type = FIRST:        return low    if type =
LAST:        return high    if type = MEDIAN:
      mid = (low + high) / 2        return
index of median among
array[low], array[mid], array[high]
```

CODE:

```python
import random
import time

def quicksort(a, pivot_type):
    if len(a) <= 1:
        return a

    if pivot_type == "first":
        pivot = a[0]

    elif pivot_type == "last":
        pivot = a[-1]

    else:
        x = [a[0], a[len(a)//2], a[-1]]
        pivot = sorted(x)[1]

    left = [x for x in a if x < pivot]
    middle = [x for x in a if x == pivot]
    right = [x for x in a if x > pivot]

    return quicksort(left, pivot_type) + middle + quicksort(right, pivot_type)


sizes = [1000, 2000, 4000, 8000]

for n in sizes:
    data = [random.randint(1, 10000) for _ in range(n)]

    print("\nInput Size:", n)

    for pivot in ["first", "last", "median"]:
        start = time.time()

        quicksort(data, pivot)

        end = time.time()

        print(pivot, "pivot time:",
```

OUTPUT:

```
Your Output

Input Size: 1000
first pivot time: 0.001338 seconds
last pivot time: 0.001478 seconds
median pivot time: 0.001301 seconds

Input Size: 2000
first pivot time: 0.002898 seconds
last pivot time: 0.002833 seconds
median pivot time: 0.002727 seconds
```

RESULT:

Thus, Quick Sort with median-of-three pivot selection provides better practical performance than fixed first-element and last-element pivot strategies, particularly for sorted, reverse-sorted, and partially ordered datasets. It reduces the probability of highly unbalanced partitions and makes Quick Sort more robust for large datasets.